



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA0905 JAVA PROGRAMMING

Deevena G

192311391

EMPLOYEE.JAVA

PACKAGE APPLICATION

```
package Application;
```

```
public class Employee {
```

```
    private int employeeId;  
    private String name;  
    private String gender;  
    private String dob;  
    private String contact;  
    private String email;  
    private String address;  
    private String department;  
    private String designation;  
    private String joiningDate;  
    private double basicSalary;  
    private double allowances;  
    private double deductions;  
    private String status;
```

```
    public Employee() {  
    }
```

```
    public Employee(int employeeId, String name, String gender, String dob,  
                    String contact, String email, String address,  
                    String department, String designation, String joiningDate,  
                    double basicSalary, double allowances,  
                    double deductions, String status) {
```

```
        this.employeeId = employeeId;  
        this.name = name;  
        this.gender = gender;
```

```
this.dob = dob;
this.contact = contact;
this.email = email;
this.address = address;
this.department = department;
this.designation = designation;
this.joiningDate = joiningDate;
this.basicSalary = basicSalary;
this.allowances = allowances;
this.deductions = deductions;
this.status = status;
}

public int getEmployeeId() {
    return employeeId;
}

public void setEmployeeId(int employeeId) {
    this.employeeId = employeeId;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getGender() {
    return gender;
}

public void setGender(String gender) {
    this.gender = gender;
}

public String getDob() {
    return dob;
}

public void setDob(String dob) {
    this.dob = dob;
}
```

```
}

public String getContact() {
    return contact;
}

public void setContact(String contact) {
    this.contact = contact;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public String getDesignation() {
    return designation;
}

public void setDesignation(String designation) {
    this.designation = designation;
}

public String getJoiningDate() {
```

```

        return joiningDate;
    }

    public void setJoiningDate(String joiningDate) {
        this.joiningDate = joiningDate;
    }

    public double getBasicSalary() {
        return basicSalary;
    }

    public void setBasicSalary(double basicSalary) {
        this.basicSalary = basicSalary;
    }

    public double getAllowances() {
        return allowances;
    }

    public void setAllowances(double allowances) {
        this.allowances = allowances;
    }

    public double getDeductions() {
        return deductions;
    }

    public void setDeductions(double deductions) {
        this.deductions = deductions;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}

```

DBCONNECTION.JAVA

```

import java.sql.Connection;
import java.sql.DriverManager;

```

```

import java.sql.SQLException;

public class DBConnection {

    public static Connection getConnection() {

        Connection con = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/employee_db",
                "root",
                "kaaran2006"
            );

        } catch (ClassNotFoundException e) {
            throw new RuntimeException(e);

        } catch (SQLException e) {
            throw new RuntimeException(e);
        }

        return con;
    }
}

```

EMPLOYEEMANAGEMENT.JAVA

```

import Application.Employee;

import java.awt.*;
import java.awt.event.*;
import java.sql.ResultSet;

public class EmployeeManagement extends Frame {

    Label lblTitle, lblId, lblName, lblGender, lblDob, lblContact;
    Label lblEmail, lblAddress, lblDept, lblDesignation, lblJoining;
    Label lblBasic, lblAllowance, lblDeduction, lblStatus;

    TextField txtId, txtName, txtDob, txtContact, txtEmail;
    TextField txtAddress, txtDept, txtDesignation, txtJoining;

```

TextField txtBasic, txtAllowance, txtDeduction;

Choice genderChoice;

Choice statusChoice;

Button btnAdd, btnSearch, btnUpdate, btnDelete;

Button btnClear, btnView, btnPayroll;

TextArea txtArea;

public EmployeeManagement() {

setTitle("Employee Management and Payroll Application");

setSize(1000, 700);

setLayout(new BorderLayout(10, 10));

lblTitle = new Label("EMPLOYEE MANAGEMENT AND PAYROLL
SYSTEM", Label.CENTER);

lblTitle.setFont(new Font("Arial", Font.BOLD, 22));

add(lblTitle, BorderLayout.NORTH);

Panel formPanel = new Panel();

formPanel.setLayout(new GridLayout(8, 4, 8, 8));

lblId = new Label("Employee ID");

txtId = new TextField();

lblName = new Label("Name");

txtName = new TextField();

lblGender = new Label("Gender");

genderChoice = new Choice();

genderChoice.add("Male");

genderChoice.add("Female");

lblDob = new Label("Date of Birth");

txtDob = new TextField();

lblContact = new Label("Contact Number");

txtContact = new TextField();

lblEmail = new Label("Email");

txtEmail = new TextField();

```
lblAddress = new Label("Address");  
txtAddress = new TextField();
```

```
lblDept = new Label("Department");  
txtDept = new TextField();
```

```
lblDesignation = new Label("Designation");  
txtDesignation = new TextField();
```

```
lblJoining = new Label("Date of Joining");  
txtJoining = new TextField();
```

```
lblBasic = new Label("Basic Salary");  
txtBasic = new TextField();
```

```
lblAllowance = new Label("Allowances");  
txtAllowance = new TextField();
```

```
lblDeduction = new Label("Deductions");  
txtDeduction = new TextField();
```

```
lblStatus = new Label("Employment Status");  
statusChoice = new Choice();  
statusChoice.add("Active");  
statusChoice.add("Inactive");
```

```
formPanel.add(lblId);  
formPanel.add(txtId);  
formPanel.add(lblName);  
formPanel.add(txtName);
```

```
formPanel.add(lblGender);  
formPanel.add(genderChoice);  
formPanel.add(lblDob);  
formPanel.add(txtDob);
```

```
formPanel.add(lblContact);  
formPanel.add(txtContact);  
formPanel.add(lblEmail);  
formPanel.add(txtEmail);
```

```
formPanel.add(lblAddress);
```

```
formPanel.add(txtAddress);  
formPanel.add(lblDept);  
formPanel.add(txtDept);
```

```
formPanel.add(lblDesignation);  
formPanel.add(txtDesignation);  
formPanel.add(lblJoining);  
formPanel.add(txtJoining);
```

```
formPanel.add(lblBasic);  
formPanel.add(txtBasic);  
formPanel.add(lblAllowance);  
formPanel.add(txtAllowance);
```

```
formPanel.add(lblDeduction);  
formPanel.add(txtDeduction);  
formPanel.add(lblStatus);  
formPanel.add(statusChoice);
```

```
Panel buttonPanel = new Panel();  
buttonPanel.setLayout(new FlowLayout());
```

```
btnAdd = new Button("Add");  
btnSearch = new Button("Search");  
btnUpdate = new Button("Update");  
btnDelete = new Button("Delete");  
btnClear = new Button("Clear");  
btnView = new Button("View All");  
btnPayroll = new Button("Payroll");
```

```
buttonPanel.add(btnAdd);  
buttonPanel.add(btnSearch);  
buttonPanel.add(btnUpdate);  
buttonPanel.add(btnDelete);  
buttonPanel.add(btnClear);  
buttonPanel.add(btnView);  
buttonPanel.add(btnPayroll);
```

```
Panel centerPanel = new Panel();  
centerPanel.setLayout(new BorderLayout(10, 10));
```

```
centerPanel.add(formPanel, BorderLayout.NORTH);  
centerPanel.add(buttonPanel, BorderLayout.CENTER);
```



```

add(centerPanel, BorderLayout.CENTER);

txtArea = new TextArea();
txtArea.setEditable(false);
add(txtArea, BorderLayout.SOUTH);

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        dispose();
    }
});

btnAdd.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

        try {

            if (txtId.getText().isEmpty() ||
                txtName.getText().isEmpty() ||
                txtDob.getText().isEmpty() ||
                txtContact.getText().isEmpty() ||
                txtEmail.getText().isEmpty() ||
                txtAddress.getText().isEmpty() ||
                txtDept.getText().isEmpty() ||
                txtDesignation.getText().isEmpty() ||
                txtJoining.getText().isEmpty() ||
                txtBasic.getText().isEmpty() ||
                txtAllowance.getText().isEmpty() ||
                txtDeduction.getText().isEmpty()) {

                txtArea.setText("Please fill all mandatory fields.");
                return;
            }

            int id = Integer.parseInt(txtId.getText());
            double basic = Double.parseDouble(txtBasic.getText());
            double allowance = Double.parseDouble(txtAllowance.getText());
            double deduction = Double.parseDouble(txtDeduction.getText());

            if (basic < 0 || allowance < 0 || deduction < 0) {
                txtArea.setText("Salary values cannot be negative.");
                return;
            }
        }
    }
});

```

```

    }

    Employee employee = new Employee(
        id,
        txtName.getText(),
        genderChoice.getSelectedItem(),
        txtDob.getText(),
        txtContact.getText(),
        txtEmail.getText(),
        txtAddress.getText(),
        txtDept.getText(),
        txtDesignation.getText(),
        txtJoining.getText(),
        basic,
        allowance,
        deduction,
        statusChoice.getSelectedItem()
    );

    EmployeeDAO dao = new EmployeeDAO();

    boolean result = dao.insertEmployee(employee);

    if (result) {
        txtArea.setText("Employee added successfully!");
    } else {
        txtArea.setText("Failed to add employee.");
    }

    } catch (NumberFormatException ex) {
        txtArea.setText("Invalid numeric input.");
    } catch (Exception ex) {
        txtArea.setText("Error: " + ex.getMessage());
    }
    }
});

btnSearch.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

        try {

            int id = Integer.parseInt(txtId.getText());

```

```

EmployeeDAO dao = new EmployeeDAO();
ResultSet rs = dao.searchEmployee(id);

if (rs.next()) {

    txtName.setText(rs.getString("name"));
    genderChoice.select(rs.getString("gender"));
    txtDob.setText(rs.getString("dob"));
    txtContact.setText(rs.getString("contact"));
    txtEmail.setText(rs.getString("email"));
    txtAddress.setText(rs.getString("address"));
    txtDept.setText(rs.getString("department"));
    txtDesignation.setText(rs.getString("designation"));
    txtJoining.setText(rs.getString("joining_date"));
    txtBasic.setText(String.valueOf(rs.getDouble("basic_salary")));

txtAllowance.setText(String.valueOf(rs.getDouble("allowances")));

txtDeduction.setText(String.valueOf(rs.getDouble("deductions")));
    statusChoice.select(rs.getString("status"));

    txtArea.setText("Employee found successfully.");

} else {

    txtArea.setText("Employee not found.");
}

} catch (NumberFormatException ex) {

    txtArea.setText("Enter a valid Employee ID.");

} catch (Exception ex) {

    txtArea.setText("Error: " + ex.getMessage());
}
});

btnUpdate.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

```

```

try {

    int id = Integer.parseInt(txtId.getText());

    double basic = Double.parseDouble(txtBasic.getText());
    double allowance = Double.parseDouble(txtAllowance.getText());
    double deduction = Double.parseDouble(txtDeduction.getText());

    Employee employee = new Employee(
        id,
        txtName.getText(),
        genderChoice.getSelectedItem(),
        txtDob.getText(),
        txtContact.getText(),
        txtEmail.getText(),
        txtAddress.getText(),
        txtDept.getText(),
        txtDesignation.getText(),
        txtJoining.getText(),
        basic,
        allowance,
        deduction,
        statusChoice.getSelectedItem()
    );

    EmployeeDAO dao = new EmployeeDAO();

    boolean result = dao.updateEmployee(employee);

    if (result) {
        txtArea.setText("Employee updated successfully!");
    } else {
        txtArea.setText("Employee not found.");
    }

} catch (Exception ex) {

    txtArea.setText("Error: " + ex.getMessage());
}

});

btnDelete.addActionListener(new ActionListener() {

```

```

public void actionPerformed(ActionEvent e) {

    try {

        int id = Integer.parseInt(txtId.getText());

        EmployeeDAO dao = new EmployeeDAO();

        boolean result = dao.deleteEmployee(id);

        if (result) {

            txtArea.setText("Employee deleted successfully!");

        } else {

            txtArea.setText("Employee not found.");

        }

    } catch (Exception ex) {

        txtArea.setText("Error: " + ex.getMessage());

    }

}

});

btnClear.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        txtId.setText("");
        txtName.setText("");
        txtDob.setText("");
        txtContact.setText("");
        txtEmail.setText("");
        txtAddress.setText("");
        txtDept.setText("");
        txtDesignation.setText("");
        txtJoining.setText("");
        txtBasic.setText("");
        txtAllowance.setText("");
        txtDeduction.setText("");

        genderChoice.select(0);
    }

});

```

```

        statusChoice.select(0);

        txtArea.setText("");
    }
});

btnView.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

        try {

            EmployeeDAO dao = new EmployeeDAO();
            ResultSet rs = dao.viewAll();

            StringBuilder result = new StringBuilder();

            result.append("EMPLOYEE DETAILS\n\n");

            while (rs.next()) {

                result.append("ID: ")
                    .append(rs.getInt("employee_id"))
                    .append("\n");

                result.append("Name: ")
                    .append(rs.getString("name"))
                    .append("\n");

                result.append("Gender: ")
                    .append(rs.getString("gender"))
                    .append("\n");

                result.append("Department: ")
                    .append(rs.getString("department"))
                    .append("\n");

                result.append("Designation: ")
                    .append(rs.getString("designation"))
                    .append("\n");

                result.append("Basic Salary: ")
                    .append(rs.getDouble("basic_salary"))
                    .append("\n");
            }
        }
    }
});

```

```

        result.append("Allowances: ")
            .append(rs.getDouble("allowances"))
            .append("\n");

        result.append("Deductions: ")
            .append(rs.getDouble("deductions"))
            .append("\n");

        result.append("Status: ")
            .append(rs.getString("status"))
            .append("\n");

        result.append("-----\n");
    }

    txtArea.setText(result.toString());

} catch (Exception ex) {

    txtArea.setText("Error: " + ex.getMessage());
}
});

btnPayroll.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

        try {

            double basic = Double.parseDouble(txtBasic.getText());
            double allowance = Double.parseDouble(txtAllowance.getText());
            double deduction = Double.parseDouble(txtDeduction.getText());

            double grossSalary = basic + allowance;
            double netSalary = grossSalary - deduction;

            txtArea.setText(
                "PAYROLL DETAILS\n\n" +
                "Basic Salary : " + basic + "\n" +
                "Allowances  : " + allowance + "\n" +
                "Deductions  : " + deduction + "\n" +
                "Gross Salary : " + grossSalary + "\n" +

```

```

        "Net Salary  : " + netSalary
    );

    } catch (NumberFormatException ex) {

        txtArea.setText("Enter valid salary values.");
    }
}
});

setVisible(true);
}

public static void main(String[] args) {
    new EmployeeManagement();
}
}

```

EMPLOYEEA0.JAVA

```

import Application.Employee;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class EmployeeDAO {

    public boolean insertEmployee(Employee e) {

        String sql = "INSERT INTO employee
VALUES(?,?,?,?,?,?,?,?,?,?,?)";

        try {
            Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1, e.getEmployeeId());
            ps.setString(2, e.getName());
            ps.setString(3, e.getGender());
            ps.setString(4, e.getDob());
            ps.setString(5, e.getContact());
            ps.setString(6, e.getEmail());

```



```

        ps.setString(7, e.getAddress());
        ps.setString(8, e.getDepartment());
        ps.setString(9, e.getDesignation());
        ps.setString(10, e.getJoiningDate());
        ps.setDouble(11, e.getBasicSalary());
        ps.setDouble(12, e.getAllowances());
        ps.setDouble(13, e.getDeductions());
        ps.setString(14, e.getStatus());

        int rows = ps.executeUpdate();

        ps.close();
        con.close();

        return rows > 0;
    } catch (SQLException ex) {
        throw new RuntimeException(ex);
    }
}

public ResultSet searchEmployee(int id) {

    String sql = "SELECT * FROM employee WHERE employee_id=?";

    try {
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql);

        ps.setInt(1, id);

        return ps.executeQuery();

    } catch (SQLException ex) {
        throw new RuntimeException(ex);
    }
}

public boolean updateEmployee(Employee e) {

    String sql = "UPDATE employee SET name=?, gender=?, dob=?,
contact=?, email=?, address=?, department=?, designation=?, joining_date=?,
basic_salary=?, allowances=?, deductions=?, status=? WHERE

```

```
employee_id=?";
```

```
try {  
    Connection con = DBConnection.getConnection();  
    PreparedStatement ps = con.prepareStatement(sql);  
  
    ps.setString(1, e.getName());  
    ps.setString(2, e.getGender());  
    ps.setString(3, e.getDob());  
    ps.setString(4, e.getContact());  
    ps.setString(5, e.getEmail());  
    ps.setString(6, e.getAddress());  
    ps.setString(7, e.getDepartment());  
    ps.setString(8, e.getDesignation());  
    ps.setString(9, e.getJoiningDate());  
    ps.setDouble(10, e.getBasicSalary());  
    ps.setDouble(11, e.getAllowances());  
    ps.setDouble(12, e.getDeductions());  
    ps.setString(13, e.getStatus());  
    ps.setInt(14, e.getEmployeeId());  
  
    int rows = ps.executeUpdate();  
  
    ps.close();  
    con.close();  
  
    return rows > 0;  
} catch (SQLException ex) {  
    throw new RuntimeException(ex);  
}  
}
```

```
public boolean deleteEmployee(int id) {
```

```
    String sql = "DELETE FROM employee WHERE employee_id=?";
```

```
    try {  
        Connection con = DBConnection.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql);  
  
        ps.setInt(1, id);
```

```
        int rows = ps.executeUpdate();

        ps.close();
        con.close();

        return rows > 0;

    } catch (SQLException ex) {
        throw new RuntimeException(ex);
    }
}

public ResultSet viewAll() {

    String sql = "SELECT * FROM employee";

    try {
        Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql);

        return ps.executeQuery();

    } catch (SQLException ex) {
        throw new RuntimeException(ex);
    }
}
```

ADD -

Employee Management and Payroll Application

EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

Employee ID

120

Gender

Male

Contact Number

8148723801

Address

Banglore

Designation

Manager

Basic Salary

75000

Deductions

2000

Name

Karanya

Date of Birth

03-10-2006

Email

karanya@gmail.com

Department

IT

Date of Joining

11-10-2025

Allowances

1600

Employment Status

Active

Add

Search

Update

Delete

Clear

View All

Payroll

Employee added successfully!

2

Windows Taskbar

ENG IN

83%

02:24 PM 30-08-2026

SEARCH

Employee Management and Payroll Application

EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

Employee ID

120

Gender

Male

Contact Number

8148723801

Address

Banglore

Designation

Manager

Basic Salary

75000.0

Deductions

2000.0

Name

Karanya

Date of Birth

03-10-2006

Email

karanya@gmail.com

Department

IT

Date of Joining

11-10-2025

Allowances

1600.0

Employment Status

Active

Add

Search

Update

Delete

Clear

View All

Payroll

Employee found successfully.

2

Windows Taskbar

ENG IN

83%

02:26 PM 30-08-2026

UPDATE

Employee Management and Payroll Application

EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

Employee ID	120	Name	Karanya
Gender	Female	Date of Birth	03-10-2006
Contact Number	8148723801	Email	karanya@gmail.com
Address	Banglore	Department	IT
Designation	Manager	Date of Joining	11-10-2025
Basic Salary	75000.0	Allowances	1600.0
Deductions	2000.0	Employment Status	Active

Add

Search

Update

Delete

Clear

View All

Payroll

Employee updated successfully!

DELETE

EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

Employee ID		Name	
Gender	Male	Date of Birth	
Contact Number		Email	
Address		Department	
Designation		Date of Joining	
Basic Salary		Allowances	
Deductions		Employment Status	Active

Add Search Update Delete Clear View All Payroll

Employee Management and Payroll Application

EMPLOYEE MANAGEMENT AND PAYROLL SYSTEM

Employee ID

Gender

Male

Contact Number

Address

Designation

Basic Salary

Deductions

Name

Date of Birth

Email

Department

Date of Joining

Allowances

Employment Status

Active

Add

Search

Update

Delete

Clear

View All

Payroll

EMPLOYEE DETAILS

ID: 101

Name: Arun

Gender: Male

Department: IT

Designation: Developer

Basic Salary: 40000.0

Allowances: 5000.0

Deductions: 2000.0

Status: Active

2

ENG IN

 82%

02:27 PM
30-08-2026

